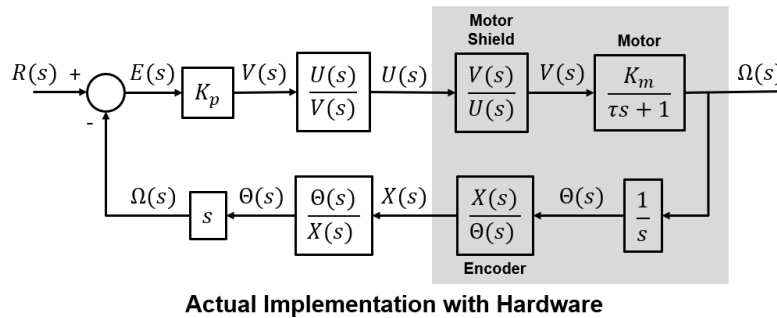
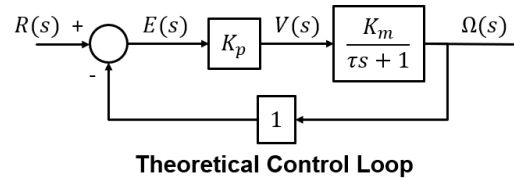




ENGS 147: Mechatronics Lab 2: Proportional Control

Spring 2026

Objective: In this lab, you build on your work from Lab 1 to implement continuous time closed-loop proportional speed control of your pancake motor. You will also consider how to accommodate the nonlinearity you observed in the PWM to voltage mapping in Lab 1.



1. Plug in your battery to ensure that it is charged. Leave your charger on the 1C setting.
2. Review your Lab 1 data and choose representative values of K_m and τ to model your motor. **Clearly state these chosen values in your lab writeup.**
3. Choose two reference input speeds (in rad/s) that are approximately 40% and 60% of the maximum motor speed that you observed in Lab 1. You don't have to hit these percentages exactly, and can use round numbers. State your chosen reference values clearly in your lab writeup. Think about (and explain in your writeup) why you aren't choosing 100% of the maximum motor speed as a reference. You might wish to simulate a closed-loop proportional controller ("theoretical" control loop as shown above) with ~ 160 rad/s reference input and examine the control effort step response as you increase the loop gain K_p .
4. Using K_m and τ from Lab 1 and the "theoretical" control loop block diagram above, choose an appropriate loop gain K_p for each of your two reference inputs. Your maximum control effort *should not exceed the maximum motor voltage* observed in Lab 1. You may use MATLAB to complete the first six columns (everything but the "experimental" section) of the table in Step 8 below.
5. **Feedback linearization.** Using the data you collected in Lab 1, derive a function $\frac{U(s)}{V(s)}$ that maps motor voltage to PWM. The idea is to "undo" the $\frac{V(s)}{U(s)}$ nonlinearity introduced by the PWM driver (motor shield), so the actual system response is more consistent with your linear model. Provide the following in your writeup:
 - A plot of your Lab 1 data with motor voltage on the x axis and PWM command on the y axis.
 - A plot of a continuous function that approximates this voltage to PWM relationship. You may overlay this on the raw data plot if you would like.
 - There are many ways to generate this function. One straightforward approach is to linearly interpolate between a set of "control points" that you will ultimately hard code into arrays. These points could be the actual voltage and PWM values from Lab 1. Alternatively, you could use `polyfit()` in MATLAB to generate a smooth approximation.

- Make sure that your function is *monotonic*, that is, that it should never switch between increasing and decreasing.
 - A C++ function that implements your continuous approximation of the voltage to PWM relationship. This function should take a voltage command as input and return the corresponding PWM. Your control loop you will call this function after computing a desired voltage. Ensure that your function will not return PWM values greater than 400 or less than -400.
 - Implement a C++ unit test program that passes ~100 motor voltage values evenly spaced between $1.2 \cdot V_{min}$ and $1.2 \cdot V_{max}$ to your function (where V_{min} and V_{max} are the minimum and maximum motor voltages observed in Lab 1) and captures the PWM values returned. Plot these data overlaid on your intended continuous function from above to ensure that the results are appropriate. You do not need to implement a control loop or command your motor in this code, just test your $\frac{U(s)}{V(s)}$ function.
6. Implement closed-loop proportional speed control for your motor, following the “actual” block diagram above. Start with a copy of your Lab 1 code and keep the loop period fixed at 5ms. You may wish to use the simple approximation $\frac{U(s)}{V(s)} = \frac{400}{V_{max}}$ for initial testing before ultimately including the function you wrote above.
7. **Data collection.** Once your control loop is working and you have included your $\frac{U(s)}{V(s)}$ function, have your code print the following data columns for the same two sets of reference input and loop gain you examined above:
- Timestamp
 - Motor speed
 - Commanded voltage
 - Commanded PWM value
8. **Analysis.** Summarize your data in the table below (use theoretical data from calculations performed above). Plot your experimental step responses on the same axes as the corresponding theoretical step responses. How closely do your experimental values match the theoretical predictions? Hypothesize on sources of error that may be affecting your measurements. Finally, using your experimental closed-loop response data, calculate updated values of the constants K_m and τ for your motor to use in Labs 3 and 4, then use these updated constants to plot a second theoretical curve on each of your step response comparison plots.

Ref. Input [rad/s]	Loop Gain K_p	Theoretical (before updating parameters)				Experimental			
		Max Control Effort [V]	Max PWM	Steady State Speed [rad/s]	Steady State Error [rad/s]	Max Control Effort [V]	Max PWM	Steady State Speed [rad/s]	Steady State Error [rad/s]

9. Writeup Guidelines and Rubric: Your writeup should include all items and analyses specified above. Make sure you include BOTH sets of K_m and τ (i.e. the ones you started with from Lab 1, and the ones you extracted later from your closed-loop response). Don't forget to include your Arduino (C++) and MATLAB code or other analysis in your PDF. Grades will be determined by assessing your combined writeup and in-person checkoff using the following rubric:

- (20 pts) Selection of K_p for two appropriate reference inputs, and description of why the reference values were chosen (see instructions)
- (20 pts) Development and unit testing of your $\frac{U(s)}{V(s)}$ mapping function
- (10 pts) Appropriate calculation of speed in rad/s in your C++ code
- (10 pts) Appropriate implementation of proportional control in your C++ code
- (20 pts) Table completed with correct calculations
- (20 pts) Analysis and plots, including correct formulas for extraction of new motor K_m and τ values from the original closed-loop responses